

```
// 手柄
Container(
  width: 40,
  height: 4,
  margin: const EdgeInsets.symmetric(vertical: 12),
  decoration: BoxDecoration(
    color: Colors.white24,
    borderRadius: BorderRadius.circular(2),
  ),
),

// 标题和关闭按钮
Padding(
  padding: const EdgeInsets.symmetric(horizontal: 16),
  child: Row(
    mainAxisAlignment: MainAxisAlignment.spaceBetween,
    children: [
      const SizedBox(width: 40), // 占位
      const Text(
        '选择修行功课',
        style: TextStyle(
          color: Colors.white,
          fontSize: 22,
          fontWeight: FontWeight.bold,
        ),
      ),
      IconButton(
        icon: const Icon(Icons.close, color: Colors.white54),
        onPressed: () => Navigator.pop(context),
      ),
    ],
  ),
),

// 重要警告提醒
Container(
  margin: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
  padding: const EdgeInsets.all(12),
  decoration: BoxDecoration(
    gradient: LinearGradient(
      colors: [
        Colors.red.withOpacity(0.2),
        Colors.orange.withOpacity(0.15),
      ],
    ),
  ),
  borderRadius: BorderRadius.circular(12),
  border: Border.all(color: Colors.orange.withOpacity(0.5), width: 1.5),
),
child: const Row(
  children: [
    Icon(Icons.warning_amber_rounded, color: Colors.orange, size: 28),
    SizedBox(width: 12),
    Expanded(
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(
            '重要提醒',
            style: TextStyle(
              color: Colors.orange,
              fontSize: 15,
              fontWeight: FontWeight.bold,
            ),
          ),
          SizedBox(height: 2),
          Text(
```

```

        '一旦确认选择，功课将永久锁定，无法修改！',
        style: TextStyle(
          color: Colors.white70,
          fontSize: 13,
        ),
      ),
    ],
  ],
),
),
],
),
),
),

const Text(
  '请选择一门深入修行的功课',
  textAlign: TextAlign.center,
  style: TextStyle(
    color: Colors.white54,
    fontSize: 14,
  ),
),

// 搜索栏
Padding(
  padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
  child: TextField(
    controller: _searchController,
    style: const TextStyle(color: Colors.white),
    decoration: InputDecoration(
      hintText: '搜索经文标题...',
      hintStyle: const TextStyle(color: Colors.white38),
      prefixIcon: const Icon(Icons.search, color: Colors.white54),
      suffixIcon: _searchQuery.isNotEmpty
        ? IconButton(
            icon: const Icon(Icons.clear, color: Colors.white54),
            onPressed: () {
              _searchController.clear();
              setState(() {
                _searchQuery = '';
                _searchResults.clear();
              });
            },
          )
        : null,
      filled: true,
      fillColor: const Color(0xFF2A2A2A),
      border: OutlineInputBorder(
        borderRadius: BorderRadius.circular(12),
        borderSide: BorderSide.none,
      ),
      contentPadding: const EdgeInsets.symmetric(horizontal: 16, vertical: ...
    ),
  ),
),

const Divider(color: Colors.white12, height: 1),

// 功课列表
Expanded(
  child: _isLoading
    ? const Center(
        child: CircularProgressIndicator(color: Color(0xFFD4AF37)),
      )
    : _searchQuery.isNotEmpty
      ? _buildSearchResults()
      : _buildGroupedList(),

```

```

),
// 底部确认按钮
Container(
  padding: const EdgeInsets.all(20),
  decoration: const BoxDecoration(
    color: Color(0xFF1E1E1E),
    border: Border(top: BorderSide(color: Colors.white12)),
  ),
  child: SafeArea(
    top: false,
    child: SizedBox(
      width: double.infinity,
      height: 56,
      child: ElevatedButton(
        onPressed: _selectedTitle != null ? _confirmSelection : null,
        style: ElevatedButton.styleFrom(
          backgroundColor: const Color(0xFFD4AF37),
          disabledBackgroundColor: Colors.white12,
          shape: RoundedRectangleBorder(
            borderRadius: BorderRadius.circular(28),
          ),
        ),
        child: Text(
          _selectedTitle != null ? '确认选择「$_selectedTitle」' : '请先选择一门功课',
          style: TextStyle(
            color: _selectedTitle != null ? Colors.black : Colors.white38,
            fontSize: 16,
            fontWeight: FontWeight.bold,
          ),
        ),
      ),
    ),
  ),
),
);
}

```

```
Widget_buildSearchResults() {
  if (_searchResults.isEmpty) {
    return Center(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.min,
        children: [
          const Icon(Icons.search_off, size: 48, color: Colors.white24),
          const SizedBox(height: 16),
          Text(
            '未找到 "$_searchQuery" 相关的经文',
            style: const TextStyle(color: Colors.white38, fontSize: 14),
          ),
        ],
      ),
    );
  }
}
```

```
return ListView.builder(
  padding: const EdgeInsets.symmetric(vertical: 8),
  itemCount: _searchResults.length,
  itemBuilder: (context, index) {
    final practice = _searchResults[index];
    return _buildPracticeItem(practice);
  },
);
}
```

```

Widget _buildGroupedList() {
  if (_groupedPractices.isEmpty) {
    return const Center(
      child: Text('暂无经文素材', style: TextStyle(color: Colors.white38)),
    );
  }

  return ListView.builder(
    padding: const EdgeInsets.symmetric(vertical: 8),
    itemCount: _groupedPractices.length,
    itemBuilder: (context, index) {
      final entry = _groupedPractices.entries.elementAt(index);
      final directory = entry.key;
      final practices = entry.value;
      final isExpanded = _expandedGroups.contains(directory);
      final dirName = directory.split('/').last;

      return Column(
        children: [
          // 分组标题
          ListTile(
            contentPadding: const EdgeInsets.symmetric(horizontal: 20),
            leading: Container(
              width: 40,
              height: 40,
              decoration: BoxDecoration(
                color: const Color(0xFF2A2A2A),
                borderRadius: BorderRadius.circular(8),
              ),
            ),
            child: Icon(
              isExpanded ? Icons.folder_open : Icons.folder,
              color: const Color(0xFFD4AF37),
              size: 20,
            ),
          ),
          title: Text(
            dirName,
            style: const TextStyle(color: Colors.white, fontWeight: FontWeight.bo...
          ),
          subtitle: Text(
            '${practices.length} 个经文',
            style: const TextStyle(color: Colors.white38, fontSize: 12),
          ),
          trailing: Icon(
            isExpanded ? Icons.expand_less : Icons.expand_more,
            color: Colors.white54,
          ),
          onTap: () {
            setState(() {
              if (isExpanded) {
                _expandedGroups.remove(directory);
              } else {
                _expandedGroups.add(directory);
              }
            });
          },
        ],
        // 展开的内容
        if (isExpanded)
          ...practices.map((p) => _buildPracticeItem(p)),
      );
    },
  );
}

```

```

Widget _buildPracticeItem(Map<String, dynamic> practice) {
  final title = practice['name'] ?? '未知';
  final filePath = practice['key'] ?? '';
  final isSelected = _selectedTitle == title && _selectedFilePath == filePath;

  return ListTile(
    contentPadding: const EdgeInsets.symmetric(horizontal: 20, vertical: 4),
    leading: Container(
      width: 44,
      height: 44,
      decoration: BoxDecoration(
        color: isSelected
          ? const Color(0xFFD4AF37).withOpacity(0.2)
          : const Color(0xFF2A2A2A),
        borderRadius: BorderRadius.circular(10),
        border: isSelected
          ? Border.all(color: const Color(0xFFD4AF37), width: 2)
          : null,
      ),
      child: Icon(
        Icons.auto_stories,
        color: isSelected ? const Color(0xFFD4AF37) : Colors.white54,
        size: 22,
      ),
    ),
    title: Text(
      title,
      style: TextStyle(
        color: isSelected ? const Color(0xFFD4AF37) : Colors.white,
        fontSize: 16,
        fontWeight: isSelected ? FontWeight.bold : FontWeight.normal,
      ),
      maxLines: 2,
      overflow: TextOverflow.ellipsis,
    ),
    trailing: isSelected
      ? const Icon(Icons.check_circle, color: Color(0xFFD4AF37), size: 26)
      : const Icon(Icons.circle_outlined, color: Colors.white24, size: 26),
    onTap: () {
      setState(() {
        _selectedTitle = title;
        _selectedFilePath = filePath;
      });
    },
  );
}

/// 显示功课选择弹窗（可关闭）
Future<void> showPracticeSelectionSheet(BuildContext context, {VoidCallback? onSelect...}) {
  return showModalBottomSheet(
    context: context,
    isScrollControlled: true,
    isDismissible: true,
    enableDrag: true,
    backgroundColor: Colors.transparent,
    builder: (context) => PracticeSelectionSheet(onSelected: onSelect),
  );
}

// =====
// 文件: lib/widgets/practice_stats_card.dart
// =====

```

```
import 'package:flutter/material.dart';
import '../services/practice_stats_service.dart';

/// 修行统计卡片 - 一念计数风格
class PracticeStatsCard extends StatefulWidget {
  final VoidCallback? onTapRecord; // 补录功课
  final VoidCallback? onTapHistory; // 查看记录
  final VoidCallback? onTapDedication; // 发愿回向
  final VoidCallback? onTapSettings; // 功课设置

  const PracticeStatsCard({
    super.key,
    this.onTapRecord,
    this.onTapHistory,
    this.onTapDedication,
    this.onTapSettings,
  });

  @override
  State<PracticeStatsCard> createState() => _PracticeStatsCardState();
}

class _PracticeStatsCardState extends State<PracticeStatsCard> {
  bool _showWeekly = true; // true=周, false=月
  final _service = PracticeStatsService();

  @override
  void initState() {
    super.initState();
    _loadData();
  }

  Future<void> _loadData() async {
    await _service.loadAllData();
    if (mounted) setState(() {});
  }

  @override
  Widget build(BuildContext context) {
    return ListenableBuilder(
      listenable: _service,
      builder: (context, _) {
        final stats = _service.stats;
        final goals = _service.goals;
        final weeklyData = _service.weeklyData;

        return Container(
          decoration: BoxDecoration(
            color: const Color(0xFF1E1E1E),
            borderRadius: BorderRadius.circular(16),
          ),
          child: Column(
            children: [
              // 顶部统计卡片 - 红色渐变背景
              _buildHeaderCard(stats),

              // 功能按钮
              _buildActionButtons(),

              // 发愿目标进度
              if (goals.isNotEmpty) _buildGoalProgress(goals.first),

              // 周/月统计
              _buildPeriodStats(weeklyData),

              // 连续打卡
            ],
          ),
        );
      },
    );
  }
}
```

```

        _buildStreakInfo(stats.consecutiveDays),
      ],
    ),
  );
},
);
}

/// 顶部红色统计卡片
Widget _buildHeaderCard(PracticeStats stats) {
  return Container(
    width: double.infinity,
    padding: const EdgeInsets.all(20),
    decoration: const BoxDecoration(
      gradient: LinearGradient(
        colors: [Color(0xFFE74C3C), Color(0xFFC0392B)],
        begin: Alignment.topLeft,
        end: Alignment.bottomRight,
      ),
      borderRadius: BorderRadius.only(
        topLeft: Radius.circular(16),
        topRight: Radius.circular(16),
      ),
    ),
    child: Column(
      children: [
        // 编辑图标和功课名
        Row(
          mainAxisAlignment: MainAxisAlignment.spaceBetween,
          children: [
            const Icon(Icons.edit_outlined, color: Colors.white54, size: 18),
            Text(
              stats.today.sutra ?? '选择功课',
              style: const TextStyle(color: Colors.white, fontSize: 16, fontWeight: ...),
            ),
            const SizedBox(width: 18),
          ],
        ),
        const SizedBox(height: 12),

        // 今日修行数
        Text(
          '${stats.today.count}',
          style: const TextStyle(
            color: Colors.white,
            fontSize: 56,
            fontWeight: FontWeight.bold,
            letterSpacing: -2,
          ),
        ),
        const Text('今日', style: TextStyle(color: Colors.white70, fontSize: 14)),

        const SizedBox(height: 20),

        // 累计统计
        Row(
          mainAxisAlignment: MainAxisAlignment.spaceEvenly,
          children: [
            _buildStatColumn('${stats.total.count}', '累计(部)'),
            _buildStatColumn('${stats.total.days}', '累计天数'),
            _buildStatColumn('${stats.consecutiveDays}', '连续天数'),
          ],
        ),

        const SizedBox(height: 16),
      ],
    ),
  );
}

```

```

// 查看报告和榜单按钮
Row(
  children: [
    Expanded(
      child: TextButton(
        onPressed: widget.onTapHistory,
        style: TextButton.styleFrom(
          foregroundColor: Colors.white,
          backgroundColor: Colors.white.withOpacity(0.1),
          padding: const EdgeInsets.symmetric(vertical: 12),
          shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular...
        ),
        child: const Text('查看报告'),
      ),
    ),
    const SizedBox(width: 12),
    Expanded(
      child: TextButton(
        onPressed: () {
          ScaffoldMessenger.of(context).showSnackBar(
            const SnackBar(content: Text('修行榜单功能开发中')),
          );
        },
        style: TextButton.styleFrom(
          foregroundColor: Colors.white,
          backgroundColor: Colors.white.withOpacity(0.1),
          padding: const EdgeInsets.symmetric(vertical: 12),
          shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular...
        ),
        child: const Text('查看榜单'),
      ),
    ),
  ],
),
],
),
],
);
}

```

```

Widget _buildStatColumn(String value, String label) {
  return Column(
    children: [
      Text(
        value,
        style: const TextStyle(
          color: Colors.white,
          fontSize: 24,
          fontWeight: FontWeight.bold,
        ),
      ),
      const SizedBox(height: 4),
      Text(label, style: const TextStyle(color: Colors.white70, fontSize: 12)),
    ],
  );
}

```

```

/// 功能按钮行
Widget _buildActionButtons() {
  return Padding(
    padding: const EdgeInsets.symmetric(vertical: 16),
    child: Row(
      mainAxisAlignment: MainAxisAlignment.spaceEvenly,
      children: [
        _buildActionButton(Icons.add_box_outlined, '补课功课', widget.onTapRecord),
        _buildActionButton(Icons.history, '查改记录', widget.onTapHistory),
        _buildActionButton(Icons.spa_outlined, '发愿回向', widget.onTapDedication),
      ],
    ),
  );
}

```



```

    _buildActionButton(Icons.settings_outlined, '功课设置', widget.onTapSettings),
  ],
),
);
}

```

```

Widget _buildActionButton(IconData icon, String label, VoidCallback? onTap) {
  return GestureDetector(
    onTap: onTap,
    child: Column(
      children: [
        Container(
          width: 48,
          height: 48,
          decoration: BoxDecoration(
            color: const Color(0xFF2A2A2A),
            borderRadius: BorderRadius.circular(12),
          ),
          child: Icon(icon, color: Colors.white70, size: 22),
        ),
        const SizedBox(height: 8),
        Text(label, style: const TextStyle(color: Colors.white70, fontSize: 12)),
      ],
    ),
  );
}

```

/// 发愿目标进度

```

Widget _buildGoalProgress(PracticeGoal goal) {
  return Padding(
    padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 8),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Row(
          mainAxisAlignment: MainAxisAlignment.spaceBetween,
          children: [
            Text(
              '发愿目标: ${goal.currentCount} / ${_formatNumber(goal.targetCount)}',
              style: const TextStyle(color: Colors.white, fontSize: 14),
            ),
            Text(
              '${goal.progress}%',
              style: const TextStyle(color: Colors.white54, fontSize: 14),
            ),
          ],
        ),
        const SizedBox(height: 8),
        ClipRRect(
          borderRadius: BorderRadius.circular(4),
          child: LinearProgressIndicator(
            value: goal.progress / 100,
            minHeight: 6,
            backgroundColor: const Color(0xFF2A2A2A),
            valueColor: const AlwaysStoppedAnimation<Color>(Color(0xFFE74C3C)),
          ),
        ),
      ],
    ),
  );
}

```

/// 周/月统计图表

```

Widget _buildPeriodStats(List<DayStats> weeklyData) {
  return Padding(
    padding: const EdgeInsets.all(16),

```

```

child: Column(
  crossAxisAlignment: CrossAxisAlignment.start,
  children: [
    Row(
      mainAxisAlignment: MainAxisAlignment.spaceBetween,
      children: [
        Text(
          '本${_showWeekly ? "周" : "月"}: ${_showWeekly ? _service.weekTotal : _s...
          style: const TextStyle(color: Colors.white, fontSize: 14),
        ),
        Container(
          decoration: BoxDecoration(
            color: const Color(0xFF2A2A2A),
            borderRadius: BorderRadius.circular(16),
          ),
          child: Row(
            children: [
              _buildPeriodToggle('周', _showWeekly, () {
                setState(() => _showWeekly = true);
                _service.fetchWeeklyStats();
              }),
              _buildPeriodToggle('月', !_showWeekly, () {
                setState(() => _showWeekly = false);
                _service.fetchMonthlyStats();
              }),
            ],
          ),
        ),
      ],
    ),
    const SizedBox(height: 16),

    // 柱状图
    SizedBox(
      height: 120,
      child: weeklyData.isEmpty
        ? const Center(child: Text('暂无数据', style: TextStyle(color: Colors.whi...
        : Row(
          mainAxisAlignment: MainAxisAlignment.spaceEvenly,
          crossAxisAlignment: CrossAxisAlignment.end,
          children: weeklyData.map((day) => _buildBar(day)).toList(),
        ),
      ),
    ],
  ),
);
}

```

```

Widget _buildPeriodToggle(String label, bool isSelected, VoidCallback onTap) {
  return GestureDetector(
    onTap: onTap,
    child: Container(
      padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 6),
      decoration: BoxDecoration(
        color: isSelected ? const Color(0xFFE74C3C) : Colors.transparent,
        borderRadius: BorderRadius.circular(16),
      ),
      child: Text(
        label,
        style: TextStyle(
          color: isSelected ? Colors.white : Colors.white54,
          fontSize: 12,
          fontWeight: isSelected ? FontWeight.bold : FontWeight.normal,
        ),
      ),
    ),
  );
}

```

```

    );
}

Widget _buildBar(DayStats day) {
  // 计算柱子高度 (相对于最大值)
  final maxCount = _service.weeklyData.map((d) => d.count).fold(0, (a, b) => a > b ...
  final ratio = maxCount > 0 ? day.count / maxCount : 0.0;
  final height = 80 * ratio + 10; // 最小高度10

  return Column(
    mainAxisAlignment: MainAxisAlignment.end,
    children: [
      Text(
        '${day.count}',
        style: TextStyle(
          color: day.count > 0 ? Colors.white : Colors.white38,
          fontSize: 11,
        ),
      ),
      const SizedBox(height: 4),
      Container(
        width: 28,
        height: height,
        decoration: BoxDecoration(
          gradient: day.count > 0
            ? const LinearGradient(
                colors: [Color(0xFFE74C3C), Color(0xFFC0392B)],
                begin: Alignment.topCenter,
                end: Alignment.bottomCenter,
              )
            : null,
          color: day.count == 0 ? const Color(0xFF2A2A2A) : null,
          borderRadius: BorderRadius.circular(4),
        ),
      ),
      const SizedBox(height: 4),
      Text(
        day.date.substring(5), // MM-DD
        style: const TextStyle(color: Colors.white38, fontSize: 10),
      ),
    ],
  );
}

```

/// 连续打卡信息

```

Widget _buildStreakInfo(int consecutiveDays) {
  return Container(
    padding: const EdgeInsets.all(16),
    decoration: const BoxDecoration(
      border: Border(top: BorderSide(color: Color(0xFF2A2A2A))),
    ),
    child: Row(
      children: [
        const Icon(Icons.local_fire_department, color: Color(0xFFE74C3C), size: 20)...
        const SizedBox(width: 8),
        Text(
          '报数记录: 您已连续打卡 $consecutiveDays 天',
          style: const TextStyle(color: Colors.white, fontSize: 14),
        ),
      ],
    ),
  );
}

```

```

String _formatNumber(int number) {
  if (number >= 10000) {

```

```
    return '${(number / 10000).toStringAsFixed(0)}万';
  }
  return number.toString();
}
}

// =====
// 文件: lib/widgets/recaptcha_dialog.dart
// =====

import 'package:flutter/material.dart';
import 'package:flutter_inappwebview/flutter_inappwebview.dart';
import '../core/config/app_config.dart';

/// Firebase手机验证对话框
/// 使用托管的HTML页面 (Firebase JS SDK) 完成reCAPTCHA验证并发送验证码
class RecaptchaDialog extends StatefulWidget {
  final String phoneNumber;

  const RecaptchaDialog({
    super.key,
    required this.phoneNumber,
  });

  /// 显示对话框，发送验证码后返回sessionInfo
  /// 返回值: {'success': true, 'verificationId': '...'} 或 {'success': false, 'error': '...'}
  static Future<Map<String, dynamic>?> show(BuildContext context, {required String ph...
    return showDialog<Map<String, dynamic>>({
      context: context,
      barrierDismissible: false,
      builder: (context) => RecaptchaDialog(phoneNumber: phoneNumber),
    });
  }

  @override
  State<RecaptchaDialog> createState() => _RecaptchaDialogState();
}

class _RecaptchaDialogState extends State<RecaptchaDialog> {
  bool _isLoading = true;
  String? _error;
  late InAppWebViewController _webViewController;

  // 使用托管的Firebase验证页面URL
  String get _verifyUrl {
    final encodedPhone = Uri.encodeComponent(widget.phoneNumber);
    // 使用Worker托管的验证页面
    return '${AppConfig.apiUrl}/phone-verify.html?phone=$encodedPhone';
  }

  @override
  Widget build(BuildContext context) {
    return Dialog(
      backgroundColor: Colors.transparent,
      child: Container(
        width: 380,
        height: 500,
        decoration: BoxDecoration(
          color: const Color(0xFF1a1a2e),
          borderRadius: BorderRadius.circular(16),
          boxShadow: [
            BoxShadow(
              color: Colors.black.withOpacity(0.5),
              blurRadius: 20,
              offset: const Offset(0, 10),
            ),
          ],
        ),
      ),
    );
  }
}
```

```

    ),
  ],
),
child: ClipRRect(
  borderRadius: BorderRadius.circular(16),
  child: Column(
    children: [
      // 标题栏
      Container(
        padding: const EdgeInsets.symmetric(horizontal: 16, vertical: 12),
        decoration: const BoxDecoration(
          border: Border(
            bottom: BorderSide(color: Colors.white10),
          ),
        ),
      ),
      child: Row(
        children: [
          const Icon(Icons.security, color: Colors.white70, size: 20),
          const SizedBox(width: 8),
          const Expanded(
            child: Text(
              '安全验证',
              style: TextStyle(color: Colors.white, fontSize: 16, fontWeigh...
            ),
          ),
          IconButton(
            icon: const Icon(Icons.close, color: Colors.white54, size: 20),
            padding: EdgeInsets.zero,
            constraints: const BoxConstraints(),
            onPressed: () => Navigator.of(context).pop({'success': false, '...
          ),
        ],
      ),
    ),
    // WebView
    Expanded(
      child: Stack(
        children: [
          InAppWebView(
            initialUrlRequest: URLRequest(url: WebUri(_verifyUrl)),
            initialSettings: InAppWebViewSettings(
              javaScriptEnabled: true,
              transparentBackground: true,
              disableContextMenu: true,
              supportZoom: false,
              userAgent: 'Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) A...
            ),
            onWebViewCreated: (controller) {
              _webViewController = controller;

              // 注册JavaScript处理器接收验证码发送结果
              controller.addJavaScriptHandler(
                handlerName: 'onCodeSent',
                callback: (args) {
                  if (args.isNotEmpty && args[0] is Map) {
                    final result = Map<String, dynamic>.from(args[0]);
                    debugPrint('验证码发送结果: $result');
                    Navigator.of(context).pop(result);
                  }
                  return null;
                },
              );
            },
            onLoadStart: (controller, url) {
              setState(() {
                _isLoading = true;

```

```

        _error = null;
      });
    },
    onLoadStop: (controller, url) {
      setState(() => _isLoading = false);
    },
    onReceivedError: (controller, request, error) {
      setState(() {
        _isLoading = false;
        _error = '加载失败: ${error.description}';
      });
    },
  ),
),
if (_isLoading)
const Center(
  child: Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: [
      CircularProgressIndicator(color: Color(0xFFFFE66D)),
      SizedBox(height: 16),
      Text('加载中...', style: TextStyle(color: Colors.white70)),
    ],
  ),
),
if (_error != null)
Center(
  child: Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: [
      const Icon(Icons.error_outline, color: Colors.redAccent, ...
      const SizedBox(height: 16),
      Text(_error!, style: const TextStyle(color: Colors.redAcc...
      const SizedBox(height: 16),
      ElevatedButton(
        onPressed: () {
          setState(() => _error = null);
          _webViewController.reload();
        },
        style: ElevatedButton.styleFrom(
          backgroundColor: const Color(0xFFFFE66D),
          foregroundColor: Colors.black,
        ),
        child: const Text('重试'),
      ),
    ],
  ),
),
],
),
),
),
),
),
),
);
}
}

```

```

// =====
// 文件: lib/widgets/reflection_dialog.dart
// =====

```

```

import 'package:flutter/material.dart';
import '../services/comment_service.dart';
import '../services/meditation_session_manager.dart';

```

```

/// 修行心得填写弹窗
///
/// 修行结束后显示，用户可填写心得并发布到对应经文的评论区
class ReflectionDialog extends StatefulWidget {
  final Duration duration;
  final int chantCount;
  final String sutraTitle;
  final String filePath;
  final VoidCallback? onCompleted;

  const ReflectionDialog({
    super.key,
    required this.duration,
    required this.chantCount,
    required this.sutraTitle,
    required this.filePath,
    this.onCompleted,
  });

  @override
  State<ReflectionDialog> createState() => _ReflectionDialogState();
}

class _ReflectionDialogState extends State<ReflectionDialog> {
  final TextEditingController _contentController = TextEditingController();
  final CommentService _commentService = CommentService();
  bool _isSubmitting = false;

  String get _formattedDuration {
    final minutes = widget.duration.inMinutes;
    final seconds = widget.duration.inSeconds % 60;
    return '${minutes.toString().padLeft(2, '0')}:${seconds.toString().padLeft(2, '0')}...
  }

  Future<void> _submitReflection() async {
    final content = _contentController.text.trim();
    if (content.isEmpty) {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(content: Text('请填写修行心得'), backgroundColor: Colors.orange),
      );
      return;
    }

    setState(() => _isSubmitting = true);

    try {
      // 构造评论内容，包含修行时长
      final fullContent = '【修行心得】$content\n\n 修行时长: $_formattedDuration | 念诵: ${widg...

      // 使用 filePath 作为 contentId
      final contentId = 'practice_${widget.filePath.hashCode.abs()}';

      final result = await _commentService.postComment(
        contentId,
        fullContent,
        contentType: widget.sutraTitle,
        filePath: widget.filePath,
        tag: 'practice', // 修行心得标签
      );

      if (mounted) {
        if (result['success'] == true) {
          Navigator.pop(context);
          widget.onCompleted?.call();
          ScaffoldMessenger.of(context).showSnackBar(

```

```

        const SnackBar(
          content: Text('心得已发布，功德无量'),
          backgroundColor: Color(0xFFD4AF37),
        ),
      );
    } else {
      ScaffoldMessenger.of(context).showSnackBar(
        SnackBar(
          content: Text(result['error'] ?? '发布失败，请稍后重试'),
          backgroundColor: Colors.red,
        ),
      );
    }
  }
} catch (e) {
  debugPrint('发布心得失败: $e');
  if (mounted) {
    ScaffoldMessenger.of(context).showSnackBar(
      const SnackBar(content: Text('网络错误，请稍后重试'), backgroundColor: Colors.red),
    );
  }
} finally {
  if (mounted) {
    setState(() => _isSubmitting = false);
  }
}
}
}

```

```

@override
void dispose() {
  _contentController.dispose();
  super.dispose();
}

```

```

@override
Widget build(BuildContext context) {
  return AlertDialog(
    backgroundColor: const Color(0xFFE1E1E1),
    shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(20)),
    title: const Row(
      children: [
        Text("", style: TextStyle(fontSize: 28)),
        SizedBox(width: 12),
        Text('功德圆满', style: TextStyle(color: Colors.white, fontSize: 22)),
      ],
    ),
    content: SingleChildScrollView(
      child: Column(
        mainAxisAlignment: MainAxisAlignment.min,
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          // 修行统计
          Container(
            padding: const EdgeInsets.all(16),
            decoration: BoxDecoration(
              gradient: LinearGradient(
                colors: [
                  const Color(0xFFD4AF37).withOpacity(0.2),
                  const Color(0xFFD4AF37).withOpacity(0.05),
                ],
                begin: Alignment.topLeft,
                end: Alignment.bottomRight,
              ),
              borderRadius: BorderRadius.circular(12),
              border: Border.all(color: const Color(0xFFD4AF37).withOpacity(0.3)),
            ),
          ),
        ],
      ),
    ),
  );
}

```



```

    child: Column(
      children: [
        _buildStatRow(' 功课', widget.sutraTitle),
        const Divider(color: Colors.white12, height: 20),
        _buildStatRow(' 修行时长', _formattedDuration),
        const Divider(color: Colors.white12, height: 20),
        _buildStatRow(' 念诵遍数', '${widget.chantCount}'),
      ],
    ),
  ),
),

const SizedBox(height: 20),

// 心得输入
const Text(
  '记录修行心得（可选）',
  style: TextStyle(color: Colors.white70, fontSize: 14),
),
const SizedBox(height: 8),
TextField(
  controller: _contentController,
  maxLines: 4,
  maxLength: 500,
  style: const TextStyle(color: Colors.white),
  decoration: InputDecoration(
    hintText: '今日修行有何感悟？与同修分享...',
    hintStyle: const TextStyle(color: Colors.white38),
    filled: true,
    fillColor: const Color(0xFF2A2A2A),
    border: OutlineInputBorder(
      borderRadius: BorderRadius.circular(12),
      borderSide: BorderSide.none,
    ),
    counterStyle: const TextStyle(color: Colors.white38),
  ),
),

const SizedBox(height: 8),
Text(
  '心得将发布到「${widget.sutraTitle}」的评论区',
  style: const TextStyle(color: Colors.white38, fontSize: 12),
),
),
),
actions: [
  TextButton(
    onPressed: _isSubmitting ? null : () {
      Navigator.pop(context);
      widget.onCompleted?.call();
    },
    child: const Text('跳过', style: TextStyle(color: Colors.white54)),
  ),
  ElevatedButton(
    onPressed: _isSubmitting ? null : _submitReflection,
    style: ElevatedButton.styleFrom(
      backgroundColor: const Color(0xFFD4AF37),
      disabledBackgroundColor: Colors.white24,
    ),
    child: _isSubmitting
      ? const SizedBox(
          width: 20,
          height: 20,
          child: CircularProgressIndicator(strokeWidth: 2, color: Colors.whit...
        )
      : const Text('发布心得', style: TextStyle(color: Colors.black)),
  ),

```

```

    ),
  ],
);
}

Widget _buildStatRow(String label, String value) {
  return Row(
    mainAxisAlignment: MainAxisAlignment.spaceBetween,
    children: [
      Text(label, style: const TextStyle(color: Colors.white70, fontSize: 14)),
      Text(
        value,
        style: const TextStyle(
          color: Color(0xFFD4AF37),
          fontWeight: FontWeight.bold,
          fontSize: 16,
        ),
      ),
    ],
  ),
];
};
}
}

```

```

/// 显示修行心得填写弹窗
Future<void> showReflectionDialog(
  BuildContext context, {
    required Duration duration,
    required int chantCount,
    required String sutraTitle,
    required String filePath,
    VoidCallback? onCompleted,
  }) {
  return showDialog(
    context: context,
    barrierDismissible: false,
    builder: (context) => ReflectionDialog(
      duration: duration,
      chantCount: chantCount,
      sutraTitle: sutraTitle,
      filePath: filePath,
      onCompleted: onCompleted,
    ),
  );
}

```

```

// =====
// 文件: lib/widgets/space_background.dart
// =====

```

```

import 'package:flutter/material.dart';
import '../core/design_system/colors.dart';

```

```

class SpaceBackground extends StatelessWidget {
  final Widget child;

  const SpaceBackground({super.key, required this.child});

  @override
  Widget build(BuildContext context) {
    return Stack(
      children: [
        // Deep Black Space Background
        Container(
          decoration: const BoxDecoration(
            color: Colors.black,

```

```

        gradient: RadialGradient(
          center: Alignment.center,
          radius: 1.5,
          colors: [
            Color(0xFF0B1026),
            Colors.black,
          ],
          stops: [0.0, 1.0],
        ),
      ),
    ),
  ),

  // Subtle Nebula Effect
  Positioned(
    top: -100,
    right: -100,
    child: Container(
      width: 500,
      height: 500,
      decoration: BoxDecoration(
        shape: BoxShape.circle,
        gradient: RadialGradient(
          colors: [
            nebulaPurple.withOpacity(0.1),
            Colors.transparent,
          ],
        ),
      ),
    ),
  ),
),

// Content
child,
],
);
}
}

// =====
// 文件: lib/widgets/sutra_dialog.dart
// =====

import 'package:flutter/material.dart';
import '../models/sutra_model.dart';

class SutraDialog extends StatefulWidget {
  final String title;

  const SutraDialog({Key? key, required this.title}) : super(key: key);

  @override
  State<SutraDialog> createState() => _SutraDialogState();
}

class _SutraDialogState extends State<SutraDialog> with SingleTickerProviderStateMixi...
  late AnimationController _controller;
  late Animation<double> _scaleAnimation;
  late Animation<double> _fadeAnimation;

  @override
  void initState() {
    super.initState();
    _controller = AnimationController(vsync: this, duration: const Duration(milliseco...

    _scaleAnimation = CurvedAnimation(parent: _controller, curve: Curves.easeOutBack)...

```

```
_fadeAnimation = CurvedAnimation(parent: _controller, curve: Curves.easeIn);

_controller.forward();
}

@override
void dispose() {
  _controller.dispose();
  super.dispose();
}

String _getSutraContent() {
  final sutra = SutraLibrary.getSutraByTitle(widget.title);
  return sutra?.content ?? '经文内容...';
}

@override
Widget build(BuildContext context) {
  return FadeTransition(
    opacity: _fadeAnimation,
    child: ScaleTransition(
      scale: _scaleAnimation,
      child: Dialog(
        backgroundColor: Colors.transparent,
        child: Container(
          constraints: const BoxConstraints(maxWidth: 600, maxHeight: 700),
          decoration: BoxDecoration(
            gradient: const LinearGradient(
              begin: Alignment.topLeft,
              end: Alignment.bottomRight,
              colors: [Color(0xFFFFF8DC), Color(0xFFFFF0DB), Color(0xFFFFE4B5)],
            ),
            borderRadius: BorderRadius.circular(20),
            border: Border.all(color: const Color(0xFFB8860B), width: 3),
            boxShadow: [
              BoxShadow(color: Colors.black.withOpacity(0.3), blurRadius: 20, spread: 0),
            ],
          ),
          child: Column(
            children: [
              // 标题栏
              Container(
                padding: const EdgeInsets.all(20),
                decoration: BoxDecoration(
                  gradient: LinearGradient(
                    colors: [const Color(0xFFDAA520), const Color(0xFFB8860B)],
                  ),
                  borderRadius: const BorderRadius.only(
                    topLeft: Radius.circular(17),
                    topRight: Radius.circular(17),
                  ),
                ),
              ),
              child: Row(
                children: [
                  const Icon(Icons.auto_stories, color: Colors.white, size: 32),
                  const SizedBox(width: 12),
                  Expanded(
                    child: Text(
                      widget.title,
                      style: const TextStyle(
                        fontSize: 28,
                        fontWeight: FontWeight.bold,
                        color: Colors.white,
                        shadows: [
                          Shadow(color: Colors.black26, blurRadius: 4, offset: Of...
```

```

    ],
  ),
),
IconButton(
  icon: const Icon(Icons.close, color: Colors.white),
  onPressed: () => Navigator.pop(context),
),
],
),
),
),

// 装饰线
Container(
  height: 3,
  decoration: BoxDecoration(
    gradient: LinearGradient(
      colors: [Colors.transparent, const Color(0xFFFFD700), Colors.tr...
    ),
  ),
),

// 经文内容
Expanded(
  child: Container(
    padding: const EdgeInsets.all(24),
    child: SingleChildScrollView(
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          // 经文分类标签
          Container(
            padding: const EdgeInsets.symmetric(horizontal: 12, verti...
            decoration: BoxDecoration(
              color: const Color(0xFFDAA520).withOpacity(0.2),
              borderRadius: BorderRadius.circular(12),
              border: Border.all(color: const Color(0xFFDAA520), widt...
            ),
            child: Text(
              SutraLibrary.getSutraByTitle(widget.title)?.category ??...
              style: const TextStyle(
                fontSize: 14,
                color: Color(0xFF8B4513),
                fontWeight: FontWeight.bold,
              ),
            ),
          ),
        ],
      ),
      const SizedBox(height: 20),

      // 经文正文
      Text(
        _getSutraContent(),
        style: const TextStyle(
          fontSize: 18,
          height: 2.0,
          color: Color(0xFF3E2723),
          letterSpacing: 1.5,
          fontWeight: FontWeight.w500,
        ),
      ),
    ),
    const SizedBox(height: 30),

    // 底部装饰
    Center(
      child: Container(

```



```

    ),
  ),
),
);
}

Widget _buildActionButton({
  required IconData icon,
  required String label,
  required VoidCallback onPressed,
}) {
  return ElevatedButton.icon(
    onPressed: onPressed,
    icon: Icon(icon, size: 20),
    label: Text(label),
    style: ElevatedButton.styleFrom(
      backgroundColor: const Color(0xFFDAA520),
      foregroundColor: Colors.white,
      padding: const EdgeInsets.symmetric(horizontal: 20, vertical: 12),
      shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(12)),
      elevation: 3,
    ),
  );
}
}

```

```

// =====
// 文件: lib/widgets/trajectoryPainter.dart
// =====

```

```

import 'package:flutter/material.dart';
import 'dart:math' as math;
import 'package:vector_math/vector_math_64.dart' as v;

```

```

class TrajectoryPoint {
  final v.Vector3 position;
  final double timestamp;

  TrajectoryPoint(this.position, this.timestamp);
}

```

```

class TrajectoryPainter extends CustomPainter {
  final double animationValue;
  final v.Vector3 start;
  final v.Vector3 end;
  final v.Vector3 control;
  final Color color;
  final int trailLength;
  final Function(v.Vector3, Size) projectToScreen;

```

```

  TrajectoryPainter({
    required this.animationValue,
    required this.start,
    required this.end,
    required this.control,
    required this.projectToScreen,
    this.color = Colors.cyan,
    this.trailLength = 15,
  });

```

```

  @override
  void paint(Canvas canvas, Size size) {
    if (animationValue <= 0) return;

```

```

    // 计算当前彗星头部位置

```

```

    final currentPos = _getQuadraticBezierPoint(start, control, end, animationValue);
    final screenPos = projectToScreen(currentPos, size);

    // 绘制拖尾
    _drawTrail(canvas, size);

    // 绘制彗星头部
    _drawHead(canvas, screenPos);
}

void _drawTrail(Canvas canvas, Size size) {
  for (int i = 1; i <= trailLength; i++) {
    final t = animationValue - (i * 0.015);
    if (t < 0) break;

    final trailPos = _getQuadraticBezierPoint(start, control, end, t);
    final trailScreenPos = projectToScreen(trailPos, size);

    final opacity = 1.0 - (i / trailLength);
    final radius = 4.0 * opacity;

    final paint = Paint()
      ..color = color.withOpacity(opacity * 0.6)
      ..maskFilter = const MaskFilter.blur(BlurStyle.normal, 3.0);

    canvas.drawCircle(trailScreenPos, radius, paint);
  }
}

void _drawHead(Canvas canvas, Offset screenPos) {
  // 外层辉光
  final glowPaint = Paint()
    ..color = color.withOpacity(0.3)
    ..maskFilter = const MaskFilter.blur(BlurStyle.normal, 8.0);
  canvas.drawCircle(screenPos, 8.0, glowPaint);

  // 中层光晕
  final midPaint = Paint()
    ..color = color.withOpacity(0.7)
    ..maskFilter = const MaskFilter.blur(BlurStyle.normal, 4.0);
  canvas.drawCircle(screenPos, 5.0, midPaint);

  // 核心亮点
  final corePaint = Paint()..color = Colors.white;
  canvas.drawCircle(screenPos, 2.5, corePaint);
}

v.Vector3 _getQuadraticBezierPoint(v.Vector3 p0, v.Vector3 p1, v.Vector3 p2, double...
  final u = 1 - t;
  final tt = t * t;
  final uu = u * u;

  return (p0 * uu) + (p1 * (2 * u * t)) + (p2 * tt);
}

@override
bool shouldRepaint(covariant TrajectoryPainter oldDelegate) {
  return oldDelegate.animationValue != animationValue;
}
}

// =====
// 文件: lib/widgets/transfer_control_bar.dart
// =====

```



```

import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../models/file_transfer_model.dart';

class TransferControlBar extends StatelessWidget {
  final VoidCallback onPauseResume;
  final VoidCallback onCancel;

  TransferControlBar({required this.onPauseResume, required this.onCancel});

  @override
  Widget build(BuildContext context) {
    final model = Provider.of<FileTransferModel>(context);
    final status = model.status;

    return Container(
      height: 60,
      decoration: BoxDecoration(
        color: Colors.white,
        borderRadius: BorderRadius.circular(12),
        boxShadow: [
          BoxShadow(color: Colors.black.withOpacity(0.1), blurRadius: 8, offset: Offs...
        ],
      ),
      child: Row(
        mainAxisAlignment: MainAxisAlignment.spaceEvenly,
        children: [
          // 暂停/继续按钮
          if (status == TransferStatus.transferring || status == TransferStatus.pause...
            _buildControlButton(
              context: context,
              icon: status == TransferStatus.transferring ? Icons.pause : Icons.play_...
              label: status == TransferStatus.transferring ? '暂停' : '继续',
              color: status == TransferStatus.transferring ? Colors.orange : Colors.g...
              onPressed: onPauseResume,
            ),

          // 取消按钮
          if (status != TransferStatus.completed)
            _buildControlButton(
              context: context,
              icon: Icons.cancel,
              label: '取消',
              color: Colors.red,
              onPressed: onCancel,
            ),

          // 完成按钮
          if (status == TransferStatus.completed)
            _buildControlButton(
              context: context,
              icon: Icons.check_circle,
              label: '完成',
              color: Colors.green,
              onPressed: () {
                Navigator.of(context).pop();
              },
            ),
        ],
      ),
    );
  }

  // 构建控制按钮
  Widget _buildControlButton({
    required BuildContext context,

```

```
required IconData icon,
required String label,
required Color color,
required VoidCallback onPressed,
}) {
  return TextButton.icon(
    icon: Icon(icon, color: color, size: 24),
    label: Text(
      label,
      style: TextStyle(color: color, fontWeight: FontWeight.bold),
    ),
    onPressed: onPressed,
    style: TextButton.styleFrom(
      padding: EdgeInsets.symmetric(horizontal: 16, vertical: 8),
      shape: RoundedRectangleBorder(
        borderRadius: BorderRadius.circular(8),
        side: BorderSide(color: color.withOpacity(0.5)),
      ),
      backgroundColor: color.withOpacity(0.1),
    ),
  );
}
```

```
// =====
// 文件: lib/widgets/transfer_control_panel.dart
// =====
```

```
import 'package:flutter/material.dart';
import 'package:file_picker/file_picker.dart';
```

```
/// 传输控制面板
```

```
///
```

```
/// 提供文件选择和传输控制功能的组件。
```

```
class TransferControlPanel extends StatelessWidget {
```

```
  /// 选择文件的回调
```

```
  final VoidCallback onPickFiles;
```

```
  /// 发送文件的回调
```

```
  final VoidCallback onSendFiles;
```

```
  /// 已选择的文件列表
```

```
  final List<PlatformFile> selectedFiles;
```

```
  /// 是否正在发送
```

```
  final bool isSending;
```

```
  /// 是否禁用发送按钮
```

```
  final bool disableSend;
```

```
  /// 构造函数
```

```
  const TransferControlPanel({
```

```
    Key? key,
```

```
    required this.onPickFiles,
```

```
    required this.onSendFiles,
```

```
    required this.selectedFiles,
```

```
    this.isSending = false,
```

```
    this.disableSend = false,
```

```
  }) : super(key: key);
```

```
@override
```

```
Widget build(BuildContext context) {
```

```
  return Card(
```

```
    child: Padding(
```

```
      padding: const EdgeInsets.all(16.0),
```

```

child: Column(
  crossAxisAlignment: CrossAxisAlignment.start,
  children: [
    Text('传输控制', style: Theme.of(context).textTheme.titleLarge),
    const SizedBox(height: 16.0),
    Row(
      children: [
        ElevatedButton.icon(
          icon: const Icon(Icons.file_upload),
          label: const Text('选择文件'),
          onPressed: isSending ? null : onPickFiles,
        ),
        const SizedBox(width: 16.0),
        Expanded(
          child: Text(
            selectedFiles.isEmpty ? '未选择文件' : '已选择 ${selectedFiles.length} 个文...',
            style: Theme.of(context).textTheme.bodyMedium,
          ),
        ),
      ],
    ),
  ],
),
if (selectedFiles.isNotEmpty)
  Padding(
    padding: const EdgeInsets.only(top: 8.0),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: selectedFiles
        .map(
          (file) => Padding(
            padding: const EdgeInsets.only(top: 4.0),
            child: Text(
              '${file.name} (${_formatFileSize(file.size)})',
              style: const TextStyle(fontSize: 12.0),
            ),
          ),
        )
        .toList(),
    ),
  ),
const SizedBox(height: 16.0),
ElevatedButton.icon(
  icon: const Icon(Icons.send),
  label: const Text('发送文件'),
  onPressed: (selectedFiles.isEmpty || isSending || disableSend) ? null : ...
  style: ElevatedButton.styleFrom(minimumSize: const Size(double.infinity...
),
if (isSending)
  Padding(
    padding: const EdgeInsets.only(top: 16.0),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: const [
        Text('正在发送...'),
        SizedBox(height: 8.0),
        LinearProgressIndicator(),
      ],
    ),
  ),
],
),
);
}

```

```
/// 格式化文件大小
```

```
String _formatFileSize(int bytes) {
```

```
if (bytes < 1024) {
  return '$bytes B';
} else if (bytes < 1024 * 1024) {
  return '${(bytes / 1024).toStringAsFixed(2)} KB';
} else if (bytes < 1024 * 1024 * 1024) {
  return '${(bytes / (1024 * 1024)).toStringAsFixed(2)} MB';
} else {
  return '${(bytes / (1024 * 1024 * 1024)).toStringAsFixed(2)} GB';
}
}
}

// =====
// 文件: lib/widgets/transfer_mode_selector.dart
// =====

import 'package:flutter/material.dart';
import 'package:provider/provider.dart';
import '../models/file_transfer_model.dart';

/// 传输模式选择器
///
/// 允许用户选择不同的传输模式，如全球发送和WiFi广播
class TransferModeSelector extends StatelessWidget {
  const TransferModeSelector({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    return Consumer<FileTransferModel>(
      builder: (context, model, child) {
        return Card(
          child: Padding(
            padding: const EdgeInsets.all(16.0),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text('法布施设置', style: Theme.of(context).textTheme.titleLarge),
                const SizedBox(height: 16.0),
                _buildLoopingSwitch(
                  context,
                  title: '循环法布施',
                  subtitle: '持续重复传播选定的法布施内容',
                  value: model.isLooping,
                  onChanged: (value) {
                    model.setLooping(value);
                  },
                ),
                const Divider(),
                _buildSpeedControl(context, model),
              ],
            ),
          ),
        );
      },
    );
  }

  Widget _buildModeSwitch(
    BuildContext context, {
    required String title,
    required String subtitle,
    required bool value,
    required ValueChanged<bool> onChanged,
    bool isHighlighted = false,
  }) {
```

```
return ListTile(
  title: Text(
    title,
    style: TextStyle(
      fontWeight: isHighlighted ? FontWeight.bold : FontWeight.normal,
      color: isHighlighted ? Theme.of(context).colorScheme.primary : null,
    ),
  ),
  subtitle: Text(subtitle),
  trailing: Switch(
    value: value,
    onChanged: onChanged,
    activeColor: isHighlighted ? Theme.of(context).colorScheme.primary : null,
  ),
);
}

Widget _buildLoopingSwitch(
  BuildContext context, {
    required String title,
    required String subtitle,
    required bool value,
    required ValueChanged<bool> onChanged,
  }) {
  return ListTile(
    title: Text(title),
    subtitle: Text(subtitle),
    trailing: Switch(value: value, onChanged: onChanged, activeColor: Colors.amber)...
  );
}

Widget _buildSpeedControl(BuildContext context, FileTransferModel model) {
  return Column(
    crossAxisAlignment: CrossAxisAlignment.start,
    children: [
      Text('发送速度: ${model.sendRateMB.toStringAsFixed(1)} MB/秒'),
      Slider(
        value: model.sendRateMB,
        min: 0.1,
        max: 5.0,
        divisions: 49,
        onChanged: (value) {
          model.setSendRateMB(value);
        },
      ),
    ],
  );
}

// =====
// 文件: lib/widgets/transfer_stats_display.dart
// =====

import 'package:flutter/material.dart';

/// 传输统计显示组件
///
/// 显示文件传输的统计信息，如已发送文件数、已发送数据量等。
class TransferStatsDisplay extends StatelessWidget {
  /// 已发送文件数
  final int sentCount;

  /// 已发送数据量 (MB)
  final double dataSentInMB;
```

```

/// 是否显示标题
final bool showTitle;

/// 构造函数
const TransferStatsDisplay({
  Key? key,
  required this.sentCount,
  required this.dataSentInMB,
  this.showTitle = true,
}) : super(key: key);

@override
Widget build(BuildContext context) {
  return Card(
    child: Padding(
      padding: const EdgeInsets.all(16.0),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          if (showTitle) ...[
            Text('传输统计', style: Theme.of(context).textTheme.titleLarge),
            const SizedBox(height: 16.0),
          ],
          _buildStatRow(context, '已发送节点数:', '$sentCount'),
          const SizedBox(height: 8.0),
          _buildStatRow(context, '已发送数据:', '${dataSentInMB.toStringAsFixed(2)} MB')...
        ],
      ),
    ),
  );
}

/// 构建统计行
Widget _buildStatRow(BuildContext context, String label, String value) {
  return Row(
    children: [
      Text(label, style: const TextStyle(fontWeight: FontWeight.bold)),
      const SizedBox(width: 8.0),
      Text(
        value,
        style: TextStyle(color: Theme.of(context).primaryColor, fontWeight: FontWei...
      ),
    ],
  );
}

// =====
// 文件: lib/widgets/wifi_connection_status.dart
// =====

import 'package:flutter/material.dart';

class WifiConnectionStatus extends StatelessWidget {
  const WifiConnectionStatus({Key? key}) : super(key: key);

  @override
  Widget build(BuildContext context) {
    // 这个类是空的，作为一个占位符来解决编译错误。
    // 之后会根据需要添加具体实现。
    return const SizedBox.shrink();
  }
}

```